CHAPTER 2

Number Systems & Codes

Positional systems · conversions (integers & fractions) · BCD, Excess-3, Gray, ASCII, EBCDIC · signed numbers

2.1 Positional number systems

● LEVEL 1 · BEGINNER

What is it? In everyday life we write numbers in the **decimal** system. A digit's *position* decides its value. In 345, the 3 is worth 300, not 3. Computers use other systems (binary, octal, hexadecimal) built on the same idea.

Why do we need other systems? Computer circuits have only two states, so they work in **binary**. Long binary numbers are hard for people to read, so we shorten them using **octal** and **hexadecimal**.

Real-life analogy. Think of a car odometer. Each wheel shows 0–9 and after 9 it goes back to 0 and pushes the next wheel by one. Binary is an odometer whose wheels have only two digits, 0 and 1.

Base (radix) = how many different digits the system uses. Each position has a **weight = base^{position}**, counted from 0 at the right of the point.

System	Base	Digits used	Example
Binary	2	0, 1	1011
Octal	8	0 to 7	745
Decimal	10	0 to 9	345
Hexadecimal	16	0–9, A, B, C, D, E, F (A=10 ... F=15)	2F

```
Place values of the number 345 (base 10)

     3      4      5
     |      |      |
   10^2   10^1   10^0
   100     10     1

345  = 3x100 + 4x10 + 5x1
```

```
Place values of 1011 (base 2)

     1     0     1     1
     |     |     |     |
   2^3   2^2   2^1   2^0
    8     4     2     1

1011 = 1x8 + 0x4 + 1x2 + 1x1 = 11
```

Counting 0 to 15 in all four systems

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

EASY MEMORY
Learn this table by heart. It makes every conversion in the exam quick. Notice: 1 hex digit = 4 bits and 1 octal digit = 3 bits.

2.2 Any base → Decimal

● **LEVEL 1 · BEGINNER**

How it works. Multiply every digit by its weight and add. Positions to the right of the point have **negative powers**: base^{-1}, base^{-2} …

```
Value = sum of ( digit x base^position )
Position:   ...  3   2   1   0  .  -1   -2   -3 ...
```

Simple examples (Level 1)

1011_2 = 1×8 + 0×4 + 1×2 + 1×1 = 8 + 0 + 2 + 1 = **11**

144_8 = 1×64 + 4×8 + 4×1 = 64 + 32 + 4 = **100**

$2F_{16}$ = 2×16 + 15×1 = 32 + 15 = **47**

101.101_2 = 4 + 0 + 1 + 1/2 + 0 + 1/8 = 5 + 0.5 + 0.125 = **5.625**

$0.A_{16}$ = 10 × 16^{-1} = 10/16 = **0.625**

2.3 Decimal → Any base

● LEVEL 1 · BEGINNER

Split the number at the point. Treat the **integer part** and the **fraction part** separately.

```
INTEGER part  : divide by the base again and again.
                Write down each REMAINDER.  Read remainders  BOTTOM -> TOP.

FRACTION part : multiply by the base again and again.
                Write down each INTEGER part that appears.  Read  TOP -> BOTTOM.
```

Example 1: 13 → binary (integer)

13 ÷ 2 = 6 remainder **1**

6 ÷ 2 = 3 remainder **0**

3 ÷ 2 = 1 remainder **1**

1 ÷ 2 = 0 remainder **1** → read upward: $\mathbf{1101_2}$

Example 2: 0.625 → binary (fraction)

0.625 × 2 = 1.25 → integer part **1**

0.25 × 2 = 0.5 → integer part **0**

0.5 × 2 = 1.0 → integer part **1** (fraction became 0, stop)

Read downward: $\mathbf{0.101_2}$. So $25.625_{10} = \mathbf{11001.101_2}$

Example 3: other bases

100 → octal: 100÷8 = 12 r **4**; 12÷8 = 1 r **4**; 1÷8 = 0 r **1** → $\mathbf{144_8}$

255 → hex: 255÷16 = 15 r **15**; 15÷16 = 0 r **15** → F, F → $\mathbf{FF_{16}}$

0.375 → octal: 0.375×8 = 3.0 → $\mathbf{0.3_8}$

EXAM TRAP

Many fractions never end. $\mathbf{0.1_{10}}$ in binary is 0.0001 1001 1001 … (repeating). If the question says “up to 4 places”, stop there. A fraction ends only if, after removing common factors, its denominator is a power of the base.

COMMON MISTAKE

Reading the remainders from top to bottom. For integers always read **bottom to top**.

Converting between two non-decimal bases

Two ways: (1) go through decimal, or (2) go through binary (best for octal ↔ hex). Example: 47_8 → hex. $47_8 = 100\ 111_2 = 0010\ 0111_2 = \mathbf{27_{16}}$.

2.4 Binary ↔ Octal ↔ Hexadecimal (grouping trick)

● LEVEL 2 · EXAM

Why it works. $8 = 2^3$ and $16 = 2^4$. So one octal digit is exactly 3 bits and one hex digit is exactly 4 bits.

```
Binary -> Octal :  group bits in 3s      Binary -> Hex :  group bits in 4s
Integer part   : start at the point, go LEFT,  pad zeros on the LEFT
Fraction part  : start at the point, go RIGHT, pad zeros on the RIGHT
```

Examples

101101_2 → octal: 101 | 101 → 5 5 → **55_8**

10110110_2 → hex: 1011 | 0110 → B 6 → **$B6_{16}$**

1101.0101_2 → hex: 1101 . 0101 → **$D.5_{16}$**

101.11_2 → octal: 101 . 110 (pad a zero on the right) → **5.6_8**

$7A_{16}$ → binary: 7 = 0111, A = 1010 → **1111010_2**

25.375_{10} = 11001.011_2 = 011 001 . 011 = **31.3_8** = 0001 1001 . 0110 = **19.6_{16}**

GATE POINT

GATE loves questions on **unknown base**. Set up an equation in r. Example: **$(123)_r = 83_{10}$** → $r^2 + 2r + 3 = 83$ → $r^2 + 2r - 80 = 0$ → r = 8. Check: digits used (1,2,3) must be smaller than r.

COMMON MISTAKE

Writing a digit that does not exist in the base, for example 189_8 or 1012_2. The biggest digit is always **base − 1**.

2.5 How many digits do I need?

● **LEVEL 2 · EXAM**

```
n-bit unsigned number : smallest 0, largest 2^n - 1, total 2^n values
Bits needed for N     : ceil( log2 (N + 1) )    (N = largest value to store)
n bits in octal       : ceil(n / 3) digits      n bits in hex : ceil(n / 4) digits
```

Example (Level 2)

Given: store any number up to 1000.

Concept: need $2^n - 1 \geq 1000$.

Calculation: $2^9 - 1 = 511$ (too small); $2^{10} - 1 = 1023$ (enough).

Answer: 10 bits. (Also: 10 bits need ⌈10/4⌉ = 3 hex digits.)

2.6 Codes

A **code** is an agreed way to represent numbers, letters or symbols using bits. Different codes exist because each one is good for a different job.

BCD (Binary Coded Decimal)

● **LEVEL 1 · BEGINNER**

Idea: convert **each decimal digit separately** into its 4-bit binary value (weights 8-4-2-1). Only 0000 to 1001 are valid; **1010 to 1111 are invalid** in BCD.

Example

25_{10} in BCD: 2 → 0010, 5 → 0101 → **0010 0101**

25_{10} in pure binary: **11001**. They are *different*.

BCD **1001 0111** → 9, 7 → decimal **97**.

COMMON MISTAKE

BCD is NOT the same as ordinary binary. BCD needs more bits (2 digits → 8 bits) but converts to/from decimal easily.

Excess-3 code

● LEVEL 2 · EXAM

Excess-3 = BCD + 0011 (add 3 to every digit).

Decimal	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
Excess-3	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

Self-complementing: flipping all bits of the Excess-3 code of a digit gives the Excess-3 code of its **9's complement**. Example: 4 → 0111; flip → 1000 = code of 5, and 9 − 4 = 5. This makes subtraction easier in old decimal machines. It is a **non-weighted** code.

Gray code

● LEVEL 2 · EXAM

Idea: when we go from one number to the next, **only one bit changes**. **Why useful?** In position sensors, if two bits change at the same instant and one is a little late, a wrong reading appears. With Gray code only one bit changes, so the error is at most one step.

Decimal	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111
Gray	000	001	011	010	110	111	101	100

```
BINARY -> GRAY : copy the MSB.  Every next Gray bit = this binary bit XOR the
                 binary bit on its LEFT.
GRAY -> BINARY : copy the MSB.  Every next binary bit = the binary bit just
                 found (on its left) XOR this Gray bit.
```

Examples

Binary 1011 → Gray: 1 | 1⊕0 = 1 | 0⊕1 = 1 | 1⊕1 = 0 → **1110**

Gray 1110 → Binary: 1 | 1⊕1 = 0 | 0⊕1 = 1 | 1⊕0 = 1 → **1011**

GATE POINT

Gray code is **not weighted** and is a **cyclic** code: the last code (100) differs from the first (000) in one bit too. It is the code used on K-map axes (00, 01, 11, 10).

ASCII and EBCDIC

● LEVEL 1 · BEGINNER

	ASCII	EBCDIC
Full name	American Standard Code for Information Interchange	Extended Binary Coded Decimal Interchange Code
Bits	7 bits (128 characters); extended ASCII uses 8 bits	8 bits (256 characters)
Used by	Almost all modern computers, the internet	IBM mainframes
Examples	'0' = 48, 'A' = 65 (1000001), 'a' = 97	Letters are not in one continuous block

EASY MEMORY

ASCII → common, 7 bits. EBCDIC → IBM, 8 bits. Uppercase and lowercase letters in ASCII differ by 32 (only bit 5 changes): 'A' = 65, 'a' = 97.

2.7 Signed numbers and complements

● LEVEL 2 · EXAM

Problem: we cannot write a minus sign in hardware, only 0s and 1s. **Solution:** use the **leftmost bit (MSB) as the sign bit**: 0 = positive, 1 = negative. There are three ways to store the rest.

Complements first

```
1's complement : flip every bit              101100 -> 010011
2's complement : 1's complement + 1          101100 -> 010011 + 1 = 010100

SHORTCUT for 2's complement:
   start from the RIGHT, copy bits up to and including the FIRST 1,
   then flip all the remaining bits on the left.
   101100 : copy '100', flip '101' -> '010'   =>  010100
```

The three representations

Method	How to write −5 in 4 bits	Range for n bits	Zeros
Sign-magnitude	Sign 1 + magnitude 101 → **1101**	$-(2^{n-1}-1)$ to $+(2^{n-1}-1)$	Two (+0, −0)
1's complement	Flip all bits of +5 (0101) → **1010**	$-(2^{n-1}-1)$ to $+(2^{n-1}-1)$	Two
2's complement	Flip and add 1 → **1011**	**-2^{n-1} to $+(2^{n-1}-1)$**	**One**

All 3-bit patterns, three meanings

Bits	000	001	010	011	100	101	110	111
Unsigned	0	1	2	3	4	5	6	7
Sign-magnitude	+0	1	2	3	−0	−1	−2	−3
1's complement	+0	1	2	3	−3	−2	−1	−0
2's complement	0	1	2	3	**−4**	−3	−2	−1

Why 2's complement wins: only one zero, one extra negative value, and addition and subtraction use the **same adder** with no special sign handling.

Value of a 2's-complement number: the MSB has a **negative** weight, -2^{n-1}. Example (4-bit): 1011 = −8 + 0 + 2 + 1 = **−5**.

```
4-bit 2's complement number line

1000  1001  1010  1011  1100  1101  1110  1111 | 0000 0001 0010 ... 0111
 -8    -7    -6    -5    -4    -3    -2    -1  |  0    1    2   ...   7
```

Sign extension and zero extension

To use more bits without changing the value: **signed** numbers → copy the sign bit into the new left positions (**sign extension**); **unsigned** numbers → add 0s on the left (**zero extension**).

Examples

1011 (−5) → 8 bits: **1111 1011** (still −5: −128+64+32+16+8+2+1 = −5 ✓)

0101 (+5) → 8 bits: **0000 0101**

Unsigned **1011 (11)** → 8 bits: **0000 1011** (zero extension; sign extension would wrongly give 251)

EXAM TRAP

2's complement of the most negative number is itself: in 4-bit, 1000 (−8) → flip 0111 + 1 = 1000. −8 has no positive partner in 4 bits (+8 does not fit).

GATE POINT

Number of distinct integers with n bits: $\mathbf{2^n}$ in 2's complement, but only $\mathbf{2^n - 1}$ in sign-magnitude and 1's complement (because +0 and −0 are the same integer).

GATE QUICK CHECK — Chapter 2

Formulas

```
Value = sum(digit x base^position)        Unsigned n-bit range : 0 .. 2^n - 1
2's complement range : -2^(n-1) .. 2^(n-1) - 1
Sign-magnitude / 1's complement range : -(2^(n-1) - 1) .. +(2^(n-1) - 1)
2's complement = 1's complement + 1       Bits needed for N = ceil(log2(N + 1))
```

Facts: octal digit = 3 bits, hex digit = 4 bits · BCD valid 0–9 only · Excess-3 = BCD + 3, self-complementing · Gray: one bit changes · ASCII 7-bit, EBCDIC 8-bit IBM · 2's complement has one zero.

Common traps: reading remainders in the wrong order · BCD vs binary · forgetting that 2's complement range is asymmetric · sign extension vs zero extension · invalid digit for the base.

Practice questions

● **LEVEL 1 · BEGINNER**

Q1. Convert 1101_2 to decimal.

Q2. Convert 45_{10} to binary.

Q3. Write the BCD code of 39.

● **LEVEL 2 · EXAM**

Q4. Convert $(10110.101)_2$ to octal and to hexadecimal.

Q5. Find the Gray code of binary 101101.

Q6. Find the 2's complement of 01101100.

● **LEVEL 3 · GATE**

Q7. What is the minimum number of bits needed to store every integer from −100 to +100 in 2's complement?

Q8. Convert $(2A.8)_{16}$ to decimal.

Q9. In base r, $(23)_r + (14)_r = (40)_r$. Find r.

Q10. The 8-bit 2's complement number 11010110 is extended to 16 bits. Write the 16-bit form and the decimal value.

Q11. Write −17 in 8-bit 2's complement.

Q12. How many different integers can be shown with 8 bits in (a) sign-magnitude, (b) 2's complement?

Answers

Q	Solution
1	8 + 4 + 0 + 1 = **13**.
2	45 = 32 + 8 + 4 + 1 → **101101_2**. (Check by division: 45,22,11,5,2,1 give remainders 1,0,1,1,0,1 → read up 101101.)
3	3 → 0011, 9 → 1001 → **0011 1001**.
4	**Octal:** integer 10110 → 010 110 = 26; fraction 101 → 5 → **26.5_8**. **Hex:** integer 10110 → 0001 0110 = 16; fraction 101 → 1010 = A → **$16.A_{16}$**. (Check: both equal 22.625.)
5	b = 1 0 1 1 0 1. g1 = 1; g2 = 1⊕0 = 1; g3 = 0⊕1 = 1; g4 = 1⊕1 = 0; g5 = 1⊕0 = 1; g6 = 0⊕1 = 1 → **111011**.
6	Shortcut: from the right, copy '100', flip the rest '01101' → '10010' → **10010100**. (Check: flip → 10010011, +1 → 10010100.)
7	**Given:** range −100..+100. **Concept:** need $2^{n-1} > 100$ (positive side needs $100 \le 2^{n-1} - 1$). **Calculation:** $2^6 = 64$ too small; $2^7 = 128$ works → $n - 1 = 7$. **Answer: 8 bits** (range −128..+127).
8	2×16 + 10 = 42; $0.8_{16} = 8/16 = 0.5$ → **42.5**.
9	2r + 3 + r + 4 = 4r → 3r + 7 = 4r → **r = 7**. Check: digit 4 < 7 ✓; in base 7, 3+4 = 10 (write 0, carry 1), 2+1+1 = 4 → 40 ✓.
10	MSB = 1, so copy 1s: **1111 1111 1101 0110**. Value: −128 + 64 + 16 + 4 + 2 = **−42** (unchanged by extension).
11	17 = 00010001 → flip 11101110 → +1 = **11101111**.
12	(a) Sign-magnitude has +0 and −0 → $2^8 - 1$ = **255** integers. (b) 2's complement → **256** integers (−128 to +127).

Next: Chapter 3 — Binary Arithmetic & Complements (addition, subtraction, multiplication, division, 2's complement arithmetic, overflow).